



University of Pisa

Department Of Computer Science

Advanced Programming

AI-Assisted Face Detector Web Application Project Report

Student:

Abderrahmane Salmi – 704608

ACADEMIC YEAR 2025/2026

1 Introduction

The goal of this project is to build a browser-based face detection application that runs entirely on the client side using JavaScript and a machine learning library. The system uses the user's webcam to detect faces in real time and displays bounding boxes directly in the browser. The project mainly focuses on evaluating the use of AI tools in software development, including how they assist in implementation, debugging, and verification of the final application.

2 Methodology

Instead of giving the AI a single large prompt to generate the entire application at once, I chose to build the project iteratively, feature by feature. This approach helped me maintain better control over code quality by reviewing smaller, more manageable pieces of code. It also made the output easier to read, understand, and verify, since each part was logically separated by functionality.

This way, I was able to use AI as an assistant to gradually build the application rather than relying on it to produce a complete system in one step. This ensured that I stayed in control of the overall design and structure, while avoiding the difficulty of dealing with a large complex code generated all at once.

The AI used was: Google Gemini

3 Design & Implementation with AI Assistance

3.1 Initial Planning

To begin the project, I wanted to establish a structured roadmap before starting to code. I provided the AI with the project requirements and asked it to generate a full work plan.

AI Prompt

"I need to build a university project called: "AI-Assisted Face Detector Web Application (In-Browser)"

Project requirements: - Build a browser-based face detection web application. - The application must run entirely client-side. - Use JavaScript and an in-browser ML library. - Access the user webcam with permission. - Detect faces in real time from the camera stream. - Draw bounding boxes around detected faces. - No images or video frames should be sent to a server.

Before writing any code, generate a detailed development work plan for the project."

AI Response

The AI generated a 5-step development plan which includes: setting up the environment, getting the webcam stream, integrating an ML library, drawing the bounding boxes on a canvas, and doing some basic optimization.

Human Evaluation & Verification

Based on my software engineering experience, the work plan made sense.

When it came to the ML library choice, the AI suggested several library options, including `face-api.js`, `MediaPipe Face Detection`, and `TensorFlow.js`.

After doing some research, I concluded that the best option for our project is **MediaPipe Face Detection** because it is fast, accurate, easy to integrate, and designed for real-time webcam use entirely in the browser. **face-api.js** is more complex and heavier, making it less efficient for simple live detection tasks since it includes extra features we don't need. **TensorFlow.js** (with `BlazeFace` model) is lightweight and fast, but requires more manual setup.

3.2 UI Design

Before coding the functionality of the app, we need a UI first, so I asked the AI to build a simple UI, I tried to include some details so the AI can generate a design close to what I had in mind.

AI Prompt

“Create a simple UI with:

- A container that holds two elements layered on top of each other: - `<video>` element to display the webcam stream - `<canvas>` element as a transparent overlay for drawing bounding boxes
- A "Start Camera" button to request webcam permission later.

CSS requirement: - The video and canvas must be perfectly aligned (same size and position). - Use a parent container with `position: relative`. - Use `position: absolute` on both video and canvas so they overlap exactly at (0,0). - Canvas should sit on top of the video. - Keep layout simple and centered.

Do not add JavaScript camera logic and face detection yet.”

AI Response

The AI generated the UI code, I tested it by copying it into VS Code and running the app on my browser. The result was as expected.

Human Evaluation & Verification

One minor thing was that the AI generated both the HTML and CSS code as one file (merged together), I asked it to make them separate files, because in my experience, that improves maintainability, scalability, and clarity.

3.3 Camera Stream Integration

Since the UI is now ready, I started implementing the app features, starting by a core functionality: requesting webcam access, handling user permissions, and piping the camera stream into the browser.

AI Response

The AI provided a script (`app.js`) that attaches an event listener to the "Start Camera" button.

Human Evaluation & Verification

I tested the application, the normal workflow worked perfectly, but I noticed some unhandled cases that required code corrections.

I asked the AI to add basic error handling for: user denying camera permission, no camera available, browser not supporting `getUserMedia`, etc.

The AI successfully added conditional checks, try catch blocks, and disabled the "Start Camera" button to avoid the user spamming it while processing.

3.4 Face Detection using ML Library

As we said before, the ML library used was Google's MediaPipe Face Detection.

This step required a lot of debugging because the AI kept generating broken code. It took several iterations, changes in AI prompts, and manual debugging before the code finally worked correctly.

Initial Prompt

“Now implement: Face Detection using ML Library 1. Use MediaPipe Face Detection in the browser. 2. Use the webcam video stream that is already working in a `<video>` element. 3. Run face detection in real time on the video. 4. Print the detected faces in the console (like position).”

AI Response

The first version of the AI-generated code failed immediately when the application started. After opening the browser console (F12), I found the following errors:

Uncaught SyntaxError: Unexpected token 'export' (index.html)

Uncaught ReferenceError: createImageTask is not defined (app.js)

Correction

The AI tried to incorrectly load MediaPipe’s `vision_bundle.mjs` using a standard HTML `<script>` tag. This caused a problem because `.mjs` files are ES Modules, which use modern JavaScript features like `import` and `export`. A standard script tag cannot understand these features, so the browser threw an error when it encountered the `export` keyword and stopped loading the file.

I gave these errors back to the AI, which then identified the issue and suggested a cleaner solution. We removed the HTML loading script and we imported the MediaPipe library directly inside `app.js` using JavaScript `import` statements.

404 Hallucination (Second Problem)

After fixing the module loading issue, the application still did not work. The button became stuck on “Loading AI Model...” and a new error appeared in the console:

```
Failed to load resource: the server responded with a status of 404 (Not Found)
Critical Failure Compiling Model Assets: Error: Failed to fetch model...
```

Correction

The AI had provided an outdated URL for the pre-trained model file (`.../blazeface_short_range...`). After investigating the issue, I discovered that Google had changed the repository structure and the correct file path actually used underscores (`.../blaze_face_short_range...`).

I reported the 404 error back to the AI and asked it to fix it, which it did, and the app ran successfully this time.

AI-Generated Code Removed Existing Logic

Although the app was finally working, I noticed another common AI issue: the updated AI-generated code removed the error handling logic that had been implemented earlier. The AI focused only on the ML functionality and unintentionally removed previously working code.

The Fix: I manually added the missing code back. A possible better approach would have been, instead of asking the AI to rewrite the entire code, it is much safer to provide the current version of the code and ask the AI to modify or extend it without removing existing functionality.

3.5 Bounding Box Rendering

The final main task was drawing real-time rectangles on the detected faces.

To do this, I asked the AI to use the canvas element that was already layered on top of the video feed. The idea was for the AI to draw rectangles onto the canvas using the face position coordinates returned by MediaPipe every frame.

AI Response

The AI generated code that worked, the overall logic followed a simple sequence:

1. Clear the previous drawings from the canvas
2. Capture the current webcam frame
3. Run the MediaPipe face detection model
4. Draw new bounding boxes
5. Repeat the process again on the next frame

Human Review

One important observation was that the AI handled multiple detected faces and drew bounding boxes on each one, instead of assuming only one face would exist, it used:

```
results.detections.forEach(...)
```

3.6 Extra Feature: Display live statistics on screen

Now let's start implementing some extra features, I wanted to display some statistics on screen, which might be useful for testing the app later.

AI Prompt

"Display simple live statistics on screen, and update them in real time. Please display: - Number of faces currently detected - Detection confidence score (percentage) - FPS (frames per second) Do not remove existing working functionality, comments, etc. Only modify/add the code needed for this new feature:"

AI Response

The code generated working code, and this time didn't remove existing code since we explicitly asked it to not to this time.

Human Review

One observation I found interesting was how the AI selected the highest confidence score instead of averaging or displaying all detections scores. This means the app always shows the strongest detection confidence currently visible on screen.

I don't really like this approach because if there are multiple faces, the other confidence scores are ignored. It might have been better and more informative if each face had its own confidence value shown on the screen.

3.7 Extra Feature: Toggle Face Detection On and Off

Another extra feature I wanted to add is allowing the user to toggle face detection on and off manually.

AI Prompt

“Allow the user to toggle face detection on and off.

Use the existing main button to controls this. When detection is ON, the button will turn detection OFF when clicked.”

Human Review

Again, the code ran without problems. This time the AI didn’t re-write the whole code, it just wrote sections of code and told me where to place them in the file, which I found interesting, because it was aware to not modify existing working code.

The AI used a simple true/false flag that controls whether face detection should run or not. It also kept the animation loop running all the time, when the detection is OFF, it only skipped the drawing and ML processing part.

I liked this design decision (keeping the loop running) because its a good practice for our app, it gives us simpler code, no start/stop bugs, and instant ON/OFF switching. On the other hand it wastes some performance away but for our app the simplicity is more important.

4 Testing

4.1 Functional Testing

I manually tested many cases like: camera permission granted, permission denied, multiple faces, no faces, etc.

The app passed all the test cases without any issues.

4.2 Cross-Browser Testing

Next I tried to test the app across different browser, which were: Brave, Chrome, Mozilla.

Then I tried to test the website’s responsiveness by using Chrome’s Dev Tools to run the app on a mobile website. The app failed this test because the UI wasn’t responsive, which is expected since the AI hard-coded the video resolution of the output video element, instead of making dynamic.

To solve this, I asked the AI to make the app fully responsive, and it did by removing the hard-coded resolution values from the CSS file and it let the video element dictate the height of the container based on its aspect ratio, while ensuring it never exceeds 100% of the user’s screen width.

5 Evaluation

5.1 Detection Accuracy

To evaluate detection accuracy, the application was tested in different real-world scenarios using live camera input.

- First, I tested it with one face in front of the camera under normal lighting. The app correctly detected the face and showed the bounding box in the right position.
- Next, I tested with more than one face. The system was able to detect multiple faces at the same time and showed separate boxes for each face.
- Then I tested a situation with no faces (just pointing the camera at a wall). In this case, the app did not show any detections, so there were no false positives.
- Finally, I tested partial and side faces by turning my head or covering part of my face. The app still detected a face most of the time.

5.2 Performance

The performance of the application was tested by checking the FPS and hardware usage during real-time face detection.

In normal lighting and optimal situations, the application was able to reach around 60 FPS, which made the video and face detection very smooth in real time. In worse conditions, such as low lighting, the FPS dropped slightly, but it never went below 30 FPS. On average, the application stayed around 50 FPS during normal use.

The browser Performance tab was also used to record and monitor the application while running. During testing, CPU and GPU usage remained stable, with no abnormal spikes or unusual behavior detected.

Overall, the application performs smoothly without causing major performance issues.

6 Privacy Notes

This application was designed to run completely in the browser without using any server-side processing. All face detection is performed locally on the user's device using JavaScript and the MediaPipe model.

The webcam is only accessed after the user gives permission, and no face images or video recordings are uploaded or stored. The camera feed is processed temporarily in memory only for real-time face detection and drawing bounding boxes.

7 Verification of AI-Generated Output

To verify the correctness of the AI-generated code, several validation methods were used during development.

First, the application was manually tested by running it and checking whether the features worked correctly.

For important parts of the implementation, like camera permission handling, the AI-generated code was compared with the official documentation to confirm that the implementation is correct.

The generated code was also manually reviewed using my own programming knowledge and experience. This included checking the logic of the detection loop, verifying variable usage, understanding how the MediaPipe model was integrated, and confirming that the application behaved correctly.

In addition, some AI-generated explanations and code suggestions were cross-checked using other AI models (e.g. ChatGPT, Claude) to compare answers and confirm correctness before using them in the final implementation.

These verification steps helped ensure that the final application was functional, correct, and reliable.

8 Project Links

- **Final Application (GitHub Pages):** <https://abderrahmane-salmi.github.io/ai-face-detector-webapp/>
- **Source Code Repository (GitHub):** <https://github.com/abderrahmane-salmi/ai-face-detector-webapp>
- **AI Chat Used During Development (Gemini):** <https://gemini.google.com/share/863d22e9>